



**FACULTAD DE  
CIENCIAS**

# **Comparación entre algoritmos de firma digital por curvas elípticas: ECDSA vs EC- KCDSA**

Una introducción a la encriptación basada en curvas elípticas

**Seguridad en Sistemas Informáticos (CC411A)**

Escuela de Ciencias de la Computación

*Mg. Manuel Alejandro Quispe Torres*

|                                 |           |                        |
|---------------------------------|-----------|------------------------|
| Ángel Aarón Flores Alberca      | 20221346A | a.flores.a@uni.pe      |
| Leonardo Alexander Chacón Roque | 20221002K | l.chacon.r@uni.pe      |
| Diego Akira García Rojas        | 20222510J | diego.garcia.r@uni.pe  |
| Guido Anthony Chipana Calderon  | 20221428H | guido.chipana.c@uni.pe |
| Diego Manuel Delgado Velarde    | 20222676E | diego.delgado.v@uni.pe |

28 de Mayo del 2026

# Índice

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>1. Introducción .....</b>                                                       | <b>5</b>  |
| <b>2. Fundamento teórico .....</b>                                                 | <b>5</b>  |
| 2.1. Nociones previas .....                                                        | 5         |
| 2.1.1. Definición de grupo .....                                                   | 5         |
| 2.1.2. Grupos finitos .....                                                        | 6         |
| 2.1.3. Ejemplos: Grupos $(\mathbb{F}_p, +)$ y $(\mathbb{F}_p^\times, \cdot)$ ..... | 6         |
| 2.1.4. Existencia de generadores .....                                             | 6         |
| 2.2. Criptosistemas basados en logaritmo discreto .....                            | 7         |
| 2.2.1. Notación para el logaritmo discreto .....                                   | 7         |
| 2.2.2. Problema del logaritmo discreto .....                                       | 7         |
| 2.2.3. DL Generación de Clave .....                                                | 8         |
| 2.2.4. DL Esquema de Cifrado y Descifrado .....                                    | 9         |
| 2.2.5. DL Esquema de firma .....                                                   | 9         |
| 2.3. Curvas elípticas sobre campos finitos .....                                   | 11        |
| 2.3.1. Definición de algebraica de una curva elíptica .....                        | 11        |
| 2.3.2. Ley de grupo .....                                                          | 11        |
| 2.3.2.1. Suma de puntos .....                                                      | 12        |
| 2.3.2.2. Propiedades del grupo abeliano .....                                      | 12        |
| 2.3.3. Orden del grupo .....                                                       | 13        |
| 2.4. Criptosistemas basados en curvas elípticas .....                              | 13        |
| 2.4.1. EC Generación de claves .....                                               | 13        |
| 2.4.2. EC Esquema de Cifrado y Descifrado .....                                    | 14        |
| 2.4.3. EC Esquema de Firmas (ECDSA) .....                                          | 15        |
| 2.4.4. EC-KCDSA Esquema de Firmas .....                                            | 16        |
| <b>3. Análisis de seguridad .....</b>                                              | <b>18</b> |
| 3.1. Seguridad <i>GMR</i> .....                                                    | 18        |
| 3.1.1. ECDSA .....                                                                 | 18        |
| 3.1.2. EC-KCDSA .....                                                              | 18        |
| 3.2. Verificación de $r$ y $s$ .....                                               | 18        |
| 3.2.1. ECDSA .....                                                                 | 18        |

|                                          |           |
|------------------------------------------|-----------|
| 3.2.2. EC-KCDSA .....                    | 18        |
| 3.3. Secreto $k$ .....                   | 19        |
| 3.3.1. ECDSA .....                       | 19        |
| 3.3.2. EC-KCDSA .....                    | 19        |
| <b>4. Implementacion en codigo .....</b> | <b>19</b> |
| 4.1. ECDSA .....                         | 19        |
| 4.2. EC-KCDSA .....                      | 19        |
| <b>5. Conclusiones .....</b>             | <b>19</b> |
| 5.1. Conclusión de resistencia .....     | 19        |

## Tabla de Figuras

|                                                                                                                                |    |
|--------------------------------------------------------------------------------------------------------------------------------|----|
| Figura (1) Representación geométrica de la suma de un punto $P$ con el elemento neutro $\infty$ sobre una curva elíptica ..... | 12 |
| Figura (2) Representación geométrica de la suma de dos puntos $P$ y $Q$ sobre una curva elíptica .....                         | 12 |
| Figura (3) Representación geométrica del doblado de un punto $P$ sobre una curva elíptica .                                    | 13 |

# 1. Introducción

La criptografía de clave asimétrica ha sido el pilar de la infraestructura digital moderna durante las últimas décadas, asegurando la confidencialidad y autenticidad de la información. En particular, los sistemas basados en el problema del logaritmo discreto sobre campos multiplicativos finitos  $(\mathbb{F}_p^\times, \cdot)$ , como DSA y ElGamal, dominaron el campo de diseños de protocolos seguros en este ámbito. Sin embargo, mantener niveles óptimos de seguridad frente al incremento de poder computacional requiere cada vez un mayor tamaño de parámetros y claves, resultando en un decrecimiento de la eficiencia de los sistemas en términos de eficiencia y velocidad de transmisión de datos.

Frente a esta necesidad, surge la Criptografía de Curvas Elípticas (ECC, por sus siglas en inglés) como una evolución del sistema criptográfico asimétrico basado en el problema del logaritmo discreto. Este sistema traslada el sistema tradicional de un campo multiplicativo a un grupo abeliano finito representado por los puntos racionales de una curva elíptica  $(E(\mathbb{F}_p^\times), +)$ . Este nuevo esquema de encriptación resulta en una dificultad significativamente mayor frente a ataques conocidos utilizando considerablemente menos recursos y tamaños de parámetros frente a sistemas tradicionales.

El informe técnico presentado ofrece un análisis comparativo de dos de los estándares de firma digital basados en curvas elípticas: el Algoritmo de Firma Digital de Curva Elíptica (ECDSA), de origen estadounidense y respaldado por el NIST, y el Algoritmo Digital Basado en Certificados de Corea del Sur (EC-KCDSA), estandarizado bajo la norma ISO/IEC 15946-2. Se explica la teoría necesaria requerida para comprender el problema del logaritmo discreto y la aplicación de curvas elípticas en la criptografía. Luego, se presentan detalladamente los algoritmos a comparar, los parámetros que requieren y los pasos que ejecutan para firmar digitalmente información. Finalmente, se presenta un examen crítico de sus vulnerabilidades y limitaciones conocidas en sus implementaciones frente a las exigencias de seguridad actuales.

## 2. Fundamento teórico

### 2.1. Nociones previas

#### 2.1.1. Definición de grupo

Se define a un **grupo**  $(G, *)$  como un conjunto  $G$  junto con una operación binaria  $*$  :  $G \times G \rightarrow G$  que satisface las siguientes propiedades:

- (i.) (Cerradura)  $\forall a, b \in G : a * b \in G$
- (ii.) (Asociatividad)  $a * (b * c) = (a * b) * c \quad \forall a, b, c \in G$
- (iii.) (Existencia del elemento neutro)  $\forall a \in G, !\exists e \in G : a * e = e * a = a$
- (iv.) (Existencia del elemento inverso)  $\forall a \in G, !\exists b \in G : a * b = b * a = e$

Se dice que el grupo  $(G, *)$  es finito, si  $G$  es finito, tal que el número de elementos de  $G$  es llamado el **orden del grupo** y se denota  $|G|$ .

$(G, *)$  se llama un **grupo abeliano**, si se satisface

- (v.) (Conmutatividad)  $a * b = b * a \quad \forall a, b \in G$

### 2.1.2. Grupos finitos

Sea  $G$  un grupo finito (con notación multiplicativa) de orden  $n$  y elemento neutro  $e$ . Para un elemento  $g \in G$ , se llama **orden de  $g$**  al entero positivo más pequeño  $t$  tal que  $g^t = e$ . Este entero siempre existe, y por el **teorema de Lagrange**, el orden de  $g$  es un divisor de  $n$ .

El conjunto  $\langle g \rangle = \{g^i : 0 \leq i \leq t-1\}$  de todas las potencias de  $g$ , es en sí mismo un grupo definido bajo la misma operación de  $G$  y se llama **subgrupo cíclico generado por  $g$** .

En notación aditiva, el orden de  $g \in G$  es el entero positivo más pequeño  $t$  tal que  $tg = e$  y  $\langle g \rangle = \{ig : 0 \leq i \leq t-1\}$ .

Si  $G$  tiene un elemento  $g$  de orden  $n$ , entonces se dice que  $G$  es un **grupo cíclico** y  $g$  es llamado **generador** de  $G$ . El número de generadores presentes en un grupo es igual a  $\varphi(n)$  donde  $\varphi$  representa la función  $\varphi$  de Euler para el total de valores **coprimos** hasta  $n$ .

### 2.1.3. Ejemplos: Grupos $(\mathbb{F}_p, +)$ y $(\mathbb{F}_p^\times, \cdot)$

Sea  $p$  un número primo. Definimos  $\mathbb{F}_p = \{0, 1, 2, \dots, p-1\}$  como el conjunto de enteros módulo  $p$ , provisto de las operaciones de suma y producto módulo  $p$ .

- El grupo  $(\mathbb{F}_p, +)$  donde  $+$  denota la suma módulo  $p$ , es un grupo abeliano de orden  $|\mathbb{F}_p| = p$ . El elemento neutro es 0 y el inverso de  $a \in \mathbb{F}_p$  es  $-a \bmod p$ .
- El conjunto  $\mathbb{F}_p^\times = \{1, 2, \dots, p-1\} = \mathbb{F}_p \setminus \{0\}$  forma un grupo abeliano  $(\mathbb{F}_p^\times, \cdot)$  bajo el producto modulo  $p$ . Su orden es  $|\mathbb{F}_p^\times| = p-1$ , el elemento neutro es 1 y el inverso de  $a \in \mathbb{F}_p^\times$  se denota  $a^{-1} \bmod p$ , tal que  $a \cdot a^{-1} \equiv 1 \bmod p$

### 2.1.4. Existencia de generadores

Retomando los grupos  $(\mathbb{F}_p, +)$  y  $(\mathbb{F}_p^\times, \cdot)$  definidos anteriormente, llamamos **campo**  $(\mathbb{F}_p, +, \cdot)$  a la estructura algebraica que los combina bajo la condición adicional de distributividad:

(v.) (Distributividad)  $a \cdot (b + c) = a \cdot b + a \cdot c \quad \forall a, b, c \in G$

**Probemos que  $(\mathbb{F}_p^\times, \cdot)$  es cíclico.**

Recordar que  $|\mathbb{F}_p^\times| = p-1$ . Para cada divisor  $d$  de  $p-1$ , sea  $\psi(d)$  el número de elementos de  $\mathbb{F}_p^\times$  de orden exactamente  $d$ . Como todo elemento de  $\mathbb{F}_p^\times$  tiene orden  $d$  para algún único divisor  $d$ , entonces  $\sum_{d|p-1} \psi(d) = p-1$ .

Ahora usando un resultado de la suma de valores de la función de Euler de divisores de un número, obtendremos que  $\sum_{d|p-1} \varphi(d) = p-1$ .

Así tendremos como resultado que:

$$\sum_{d|p-1} \psi(d) = \sum_{d|p-1} \varphi(d) = p-1 \quad (1)$$

Bastaría mostrar que  $\psi(d) \leq \varphi(d)$  para todo divisor  $d$  de  $p-1$ . Si  $\psi(d) = 0$  es inmediato ver que  $\psi(d) \leq \varphi(d)$ . Ahora si  $\psi(d) \geq 1$ , existe un elemento  $g \in \mathbb{F}_p^\times$  de orden  $d$  y los elementos de  $\langle g \rangle$  son las únicas raíces de  $f(x) = x^d - 1$  en  $\mathbb{F}_p$ , ya que  $\forall x \in \langle g \rangle, x^d \equiv 1 \bmod p$ .

En otras palabras, un polinomio  $f$  de grado  $d$  en  $\mathbb{F}_p$  tendría a lo más  $d$  raíces. Por tanto, todo elemento de orden  $d$  en  $\mathbb{F}_p^\times$  pertenece a  $\langle g \rangle$ .

Dicho conjunto  $\langle g \rangle$  es isomorfo con  $\mathbb{Z}/d\mathbb{Z}$  (grupo cíclico de orden  $d$ ), donde el elemento  $g^k$  tiene orden exactamente  $d$  si y solo si  $\gcd(k, d) = 1$ . El número de tales  $k$  en  $\{0, 1, \dots, d-1\}$  sería

igual a  $\varphi(d)$ , entonces el número de elementos de orden  $d$  en  $\langle g \rangle$  es  $\varphi(d)$ , tal que obtengamos  $\psi(d) = \varphi(d)$ .

Agrupando lo verificado, vemos que  $\psi(d) \leq \varphi(d)$  para todo divisor  $d$  de  $p - 1$ , también vimos que la sumatoria sobre dichos divisores en las funciones  $\psi$  y  $\varphi$  son iguales. Entonces esto fuerza a que  $\psi(d) = \varphi(d)$  para todo divisor  $d$  de  $p - 1$ , como  $\varphi(d) \geq 1$  siempre, se ve que  $\varphi(d) \geq 1$ , es decir que existen subgrupos para todos los divisores de  $p - 1$ .

Finalmente, si ello es cierto, existirá algún subgrupo generado por un elemento  $h$  de orden  $p - 1$ , lo que nos dice que  $(\mathbb{F}_p^\times, \cdot)$  es un **grupo cíclico** generado por  $h$ . ■

**Corolario:** Se puede determinar el número de subgrupos existentes de orden  $d$  en un grupo  $\mathbb{F}_p^\times$  como  $\varphi(d)$ , lo que significa que el número de generadores estaría dado como  $\varphi(p - 1)$ .

Análogamente se demuestra que para el grupo  $(\mathbb{F}_p, +)$  el número de generadores es  $\varphi(p) = p - 1$ .

## 2.2. Criptosistemas basados en logaritmo discreto

### 2.2.1. Notación para el logaritmo discreto

Sea  $(G, *)$  un grupo finito abeliano cíclico de orden  $n$ , con generador  $g \in G$ . Para todo  $h \in G$  existe un único entero  $x \in \{0, 1, \dots, n - 1\}$  tal que  $h = g^x$ . Definimos el **logaritmo discreto** de  $h$  en base  $g$  como ese exponente  $x$  y lo denotamos por

$$\log_g(h) = x \quad (2)$$

Intuitivamente,  $\log_g(h)$  representa el **número de veces que se aplica la operación del grupo** (componer con  $g$ ) para obtener el elemento  $h$ . En notación aditiva, si  $Q = kG$  para un generador  $G$ , escribiremos igualmente  $\log_G(Q) = k$ .

En lo que sigue, siempre que trabajemos en un grupo cíclico  $G = \langle g \rangle$ , usaremos la notación  $\log_g(h)$  para el exponente  $x$  tal que  $h = g^x$ . Cuando la base  $g$  sea clara por el contexto, abreviaremos simplemente  $\log(h)$ .

### 2.2.2. Problema del logaritmo discreto

Sea  $p$  un número primo y considérese el grupo multiplicativo  $(\mathbb{F}_p^\times, \cdot)$ , que es cíclico de orden  $p - 1$ , con generador  $g \in \mathbb{F}_p^\times$ .

El **problema del logaritmo discreto** en  $\mathbb{F}_p^\times$  se formula de la siguiente manera: dados  $p$ ,  $g$  y un elemento  $h \in \mathbb{F}_p^\times$ , encontrar el entero  $x = \log_g(h)$  tal que  $h \equiv g^x \pmod{p}$ .

La operación directa  $x \mapsto g^x \pmod{p}$  puede realizarse de forma eficiente, pero se considera que la operación inversa  $h \mapsto \log_g(h)$  es computacionalmente intratable cuando  $p$  es suficientemente grande y los parámetros se eligen de forma adecuada. La seguridad de los esquemas criptográficos basados en logaritmo discreto descansa precisamente en la dificultad de resolver este problema.

El primer sistema de logaritmo discreto (**DL**) fue un protocolo de acuerdo de clave pública propuesto por Diffie y Hellman en 1976. Desde entonces, se han propuesto numerosas variantes de estos esquemas. Abordaremos el esquema básico de cifrado de clave pública de ElGamal y el algoritmo de firma digital (**DSA**).

### 2.2.3. DL Generación de Clave

En sistemas de logaritmo discreto, un par de claves se asocia con un conjunto de parámetros de dominio público  $(p, q, g)$ . Donde  $p$  es un primo,  $q$  es un primo divisor de  $p - 1$ , y  $g \in [1, p - 1]$  con orden  $q$ .

En vistas de teoría de grupos, estamos definiendo un grupo  $(\mathbb{F}_p^\times, \cdot)$  de orden  $p - 1$ , luego definimos un orden  $q$  para encontrar un subgrupo de trabajo generado por algún  $g \in \mathbb{F}_p^\times$ .

Se demostró que el grupo  $(\mathbb{F}_p^\times)$  es **cíclico** y que cuenta con  $\varphi(p - 1)$  generadores. No solo eso, podemos usar algún generador del grupo completo  $h$  para hallar aquel del subgrupo de interés de la siguiente forma:

$$g \equiv h^{\frac{p-1}{q}} \mod p \quad (3)$$

Por el pequeño teorema de Fermat, tendríamos que:

$$g^q \equiv h^{\left(\frac{p-1}{q}\right)^q} \mod p \equiv h^{p-1} \mod p \equiv 1 \mod p \quad (4)$$

Por lo anterior el orden de  $g$  divide a  $q$ . Dado que  $q$  es primo, el orden de  $g$  solo puede ser 1 o  $q$ . El valor  $g = 1$  corresponde al elemento neutro, que genera un subgrupo trivial y no resulta útil criptográficamente, por lo que se descarta en la generación de parámetros. En consecuencia, cuando  $g \neq 1$ , su orden es necesariamente  $q$  y  $\langle g \rangle$  es el subgrupo de trabajo de orden  $q$ .

---

**Algoritmo 1.1:** Generación de parámetros de dominio **DL**

---

**Entrada:** Parámetros de seguridad  $l, t$

**Salida:** Parámetros de dominio **DL**  $(p, q, g)$

1. Seleccionar un primo  $p$  de  $l$  bits y un primo  $q$  de  $t$  bits que divida a  $p - 1$
  2. Seleccionar un elemento  $g$  de orden  $q$ 
    - 2.1 Seleccionar un valor  $h \in [1, p - 1]$  y calcular  $g = h^{\frac{p-1}{q}} \mod p$
    - 2.2 **si**  $g = 1$  **entonces** ir al paso 2.1
  3. **Regresar**  $(p, q, g)$
- 

Teniendo nuestro conjunto de parámetros, podemos proseguir para generar nuestro par de claves. Una llave privada es un entero  $x$  seleccionado uniformemente al azar en el intervalo  $[1, q - 1]$  (denotado como  $x \in_R [1, q - 1]$ ) y una llave pública de la forma  $y = g^x \mod p$ . Se presenta el siguiente algoritmo:

---

**Algoritmo 1.2:** Generación de par de llaves **DL**

---

**Entrada:** Parámetros de dominio **DL**  $(p, q, g)$

**Salida:** Llave pública  $y$  y llave privada  $x$

1. Seleccionar  $x \in_R [1, q - 1]$
  2. Calcular  $y = g^x \mod p$
  3. **Regresar**  $(y, x)$
-



#### 2.2.4. DL Esquema de Cifrado y Descifrado

Sí  $y$  es la llave pública del destinatario, entonces un texto plano  $m$  es cifrado multiplicándolo por  $y^k \bmod p$ , donde  $k$  es seleccionado aleatoriamente por el emisor. Este a su vez envía  $c_2 = my^k \bmod p$  y también  $c_1 = g^k \bmod p$  al destinatario, quien usa su llave privada para calcular:

$$c_1^x \equiv g^{kx} \equiv (g^x)^k \equiv y^k \pmod{p} \quad (5)$$

y multiplica a  $c_2$  con el inverso de  $c_1^x$  como  $c_1^{-x}$  para obtener  $m$ . Sí alguien intentara obtener  $m$  necesitaría calcular  $y^k \bmod p$  a partir de los parámetros de dominio  $(p, q, g, y, c_1)$ . Esto se denomina el problema de Diffie-Hellman (**DHP**), el cual se asume y se ha demostrado en algunos casos que es tan difícil como el **DLP**.

A continuación se presentan ambos algoritmos tanto de cifrado como descifrado:

---

##### Algoritmo 1.3: Encriptación básica por ElGamal

---

**Entrada:** Parámetros de dominio **DL**  $(p, q, g)$ , llave pública  $y$ , texto plano  $m \in [0, p - 1]$

**Salida:**  $(c_1, c_2)$

1. Selecciona  $k \in_R [1, q - 1]$
  2. Calcular  $c_1 = g^k \bmod p$
  3. Calcular  $c_2 = m \cdot y^k \bmod p$
  4. **Regresar**  $(c_1, c_2)$
- 

---

##### Algoritmo 1.4: Descryptación básica por ElGamal

---

**Entrada:** Parámetros de dominio **DL**  $(p, q, g)$ , llave privada  $x$ , par de encriptación  $(c_1, c_2)$

**Salida:** Texto plano  $m$

1. Calcular  $u = c_1^{-x} \bmod p$
  2. Calcular  $m = c_2 \cdot u \bmod p$
  3. **Regresar**  $m$
- 

#### 2.2.5. DL Esquema de firma

El algoritmo de firma digital (**DSA**) fue propuesto en 1991 por el Instituto Nacional de Estándares y Tecnología (**NIST**) de Estados Unidos y se especificó en una norma federal de procesamiento de información del gobierno estadounidense (**FIPS 186**) denominada Estándar de Firma Digital (**DSS**).

Un usuario con llave privada  $x$  firma un mensaje seleccionando un entero aleatorio  $k$  del intervalo  $[1, q - 1]$ , calculando  $T = g^k \bmod p$ ,  $r = T \bmod q$  y  $s = k^{-1}(h + xr) \bmod q$  donde  $h = H(m)$  es el resultado de aplicarle una función **hash** criptográfica al mensaje  $m$  en texto plano. La firma del usuario sobre  $m$  es el par  $(r, s)$ . Para verificar la firma, se debe comprobar que  $(r, s)$  satisface la ecuación anterior.

Notemos que la ecuación anterior de  $s$ , es equivalente a  $k \equiv s^{-1}(h + xr) \bmod q$

Elevando  $g$  a ambos lados se tiene:

$$g^k \equiv g^{s^{-1}(h+xr)} \pmod{p} \quad (6)$$

Lo cual es válido porque el orden de  $g$  es  $q$ . Para corroborar ello, partamos de expresar  $k \equiv s^{-1}(h + xr) \pmod{q}$  como  $k = s^{-1}(h + xr) + \lambda q$ .

Luego como  $g$  es de orden  $q$  se tiene que  $g^q \equiv 1 \pmod{p}$ , entonces al elevar por  $g$  a ambos lados:

$$g^k = g^{s^{-1}(h+xr)+\lambda q} = g^{s^{-1}(h+xr)} \cdot \underbrace{(g^q)^\lambda}_{\equiv 1 \pmod{p}} \equiv g^{s^{-1}(h+xr)} \pmod{p} \quad (7)$$

Por lo cual se demuestra que se obtiene:  $g^k \equiv g^{s^{-1}(h+xr)} \pmod{p}$

Ahora, distribuyendo el exponente como  $s^{-1}(h + xr) = hs^{-1} + xrs^{-1}$  y operando, obtenemos:

$$g^k \equiv g^{hs^{-1}} \cdot g^{xrs^{-1}} \pmod{p} \quad (8)$$

Dando forma  $g^{xrs^{-1}} = (g^x)^{rs^{-1}}$  y sustituyendo por la llave pública  $y = g^x \pmod{p}$ :

$$g^k \equiv g^{hs^{-1}} \cdot y^{rs^{-1}} \pmod{p} \quad (9)$$

Dado que  $T = g^k \pmod{p}$  por definición, se obtiene la siguiente congruencia:

$$T \equiv g^{hs^{-1}} y^{rs^{-1}} \pmod{p} \quad (10)$$

Por lo tanto para verificar la firme se necesita calcular  $T$  usando unicamente información publica  $(g, y, h, r, s)$  y comprobar que  $r = T \pmod{q}$

#### Algoritmo 1.5: Generación de firma DSA

**Entrada:** Parámetros de dominio **DL**  $(p, q, g)$ , llave privada  $x$ , texto plano  $m$

**Salida:** Firma  $(r, s)$

1. Seleccionar  $k \in_R [1, q - 1]$
2. Calcular  $T = g^k \pmod{p}$
3. Calcular  $r = T \pmod{q}$ . Si  $r = 0$  regresar al paso 1.
4. Calcular  $h = H(m)$
5. Calcular  $s = k^{-1}(h + xr) \pmod{q}$ . Si  $s = 0$  regresar al paso 1.
6. **Regresar**  $(r, s)$

#### Algoritmo 1.6: Verificación de firma DSA

**Entrada:** Parámetros de dominio **DL**  $(p, q, g)$ , llave pública  $y$ , texto plano  $m$ , firma  $(r, s)$

**Salida:** Aceptación o rechazo de la firma

1. Verificar que  $r$  y  $s$  sean enteros en el intervalo  $[1, q - 1]$ . Si alguna verificación falla, regresar(«Rechazar la firma»)
2. Calcular  $h = H(m)$

---

**Algoritmo 1.6:** Verificación de firma DSA

---

3. Calcular  $w = s^{-1} \bmod q$
  4. Calcular  $u_1 = hw \bmod q$  y  $u_2 = rw \bmod q$
  5. Calcular  $T = g^{u_1} y^{u_2} \bmod p$
  6. Calcular  $r' = T \bmod q$
  7. Si  $r = r'$  **Regresar**(Aceptar la firma);  
Si no, **Regresar**(Rechazar la firma).
- 

La falsificación de firmas en **DSA** requiere, esencialmente, recuperar la clave privada  $x$  a partir de la clave pública  $y = g^x$  o deducir el valor aleatorio  $k$  usado en alguna firma; ambos problemas se reducen a resolver el **DLP** en el subgrupo generado por  $g$ .

La dificultad del **DLP** depende de cómo está representado el grupo. Por ejemplo: Dos grupos del mismo orden son «iguales» como estructura abstracta (isomorfos) pero los algoritmos para resolver el **DLP** pueden tener velocidades distintas en uno u otro.

El grupo  $\mathbb{F}_p^\times$  puede ser vulnerado por algoritmos como **NFS** que resuelve el **DLP** en tiempo subexponencial.  $L_P \left[ \frac{1}{3}, 1.923 \right]$

## 2.3. Curvas elípticas sobre campos finitos

### 2.3.1. Definición de algebraica de una curva elíptica

Se define a una curva elíptica  $E$  sobre un campo  $K$  de **característica** distinta a 2 y 3 mediante una ecuación:

$$E : y^2 = x^3 + ax + b \quad (11)$$

con  $\Delta \neq 0$ , donde  $\Delta$  es el discriminante de  $E$  y es definido como  $\Delta = -16(4a^3 + 27b^2)$ . Esta condición nos garantiza que la curva sea **no singular**; geométricamente, esto significa que la curva no contenga cúspides o autointersecciones.

Decimos que  $E$  es definido sobre  $K$  porque los coeficientes  $a, b$  de su ecuación son elementos de  $K$ . Si  $E$  es definido sobre  $K$ , entonces  $E$  también está definido sobre cualquier extensión del campo de  $K$ .

Si  $L$  es alguna extensión del campo  $K$ , entonces el conjunto de puntos  $L$  —rationales en  $E$  es definido como:

$$E(L) = \{(x, y) \in L \times L : y^2 - x^3 - ax - b = 0\} \cup \{\infty\} \quad (12)$$

Donde  $\infty$  es el único punto en el infinito de la curva proyectiva. Este punto actúa como elemento identidad del grupo de la curva.

### 2.3.2. Ley de grupo

Sea  $E(K)$  el conjunto de puntos  $K$ -rationales de una curva elíptica definida sobre el campo  $K$ , junto con el punto en el infinito  $\infty$ . Sobre dicho conjunto se puede definir una operación binaria aditiva, denominada **suma de puntos**, que dados dos puntos de  $E(K)$  devuelve otro punto en  $E(K)$ .

Con esta operación, el par  $(E(K), +)$  adquiere estructura de **grupo abeliano** con elemento neutro  $\infty$ . Es esta estructura algebraica la que hace posible la construcción de sistemas cripto-gráficos basados en curvas elípticas.

### 2.3.2.1. Suma de puntos

La operación de suma se define geoméricamente. Dados dos puntos  $P = (x_1, y_1)$  y  $Q = (x_2, y_2)$  en  $E(K)$ , se traza la recta que pasa por  $P$  y  $Q$ , la cual intersecta a la curva en un tercer punto. El punto resultante  $R = P + Q$  se obtiene reflejando dicho punto de intersección sobre el eje  $x$ . En el caso particular en que  $P = Q$ , se emplea la recta tangente a la curva en  $P$ .

Algebraicamente, distinguimos los siguientes casos para  $R = (x_3, y_3)$ :

- Si  $P \neq Q$  y  $x_1 \neq x_2$ , entonces:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1 \quad (13)$$

- Si  $P = Q$  (doblado de punto), entonces:

$$\lambda = \frac{3x_1^2 + a}{2y_1}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1 \quad (14)$$

- Si  $x_1 = x_2$  pero  $y_1 \neq y_2$ , entonces  $P + Q = \infty$ .

### 2.3.2.2. Propiedades del grupo abeliano

A continuación se detallan las propiedades que verifican la denominación de grupo abeliano:

- (Cerradura)  $\forall P, Q \in E(K)$ , se cumple que  $P + Q \in E(K)$ .
- (Existencia de elemento neutro)  $P + \infty = \infty + P = P \quad \forall P \in E(K)$ .
- (Existencia de inverso) Para todo punto  $P = (x, y) \in E(K)$ , su inverso es  $-P = (x, -y) \in E(K)$ , tal que  $P + (-P) = \infty$ . Se tiene además que  $\infty = \infty$ .
- (Asociatividad)  $(P + Q) + R = P + (Q + R) \quad \forall P, Q, R \in E(K)$ .
- (Conmutatividad)  $P + Q = Q + P \quad \forall P, Q \in E(K)$ .

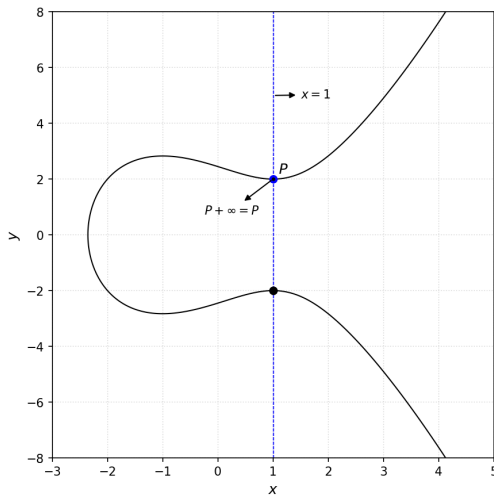


Figura (1): Representación geométrica de la suma de un punto  $P$  con el elemento neutro  $\infty$  sobre una curva elíptica

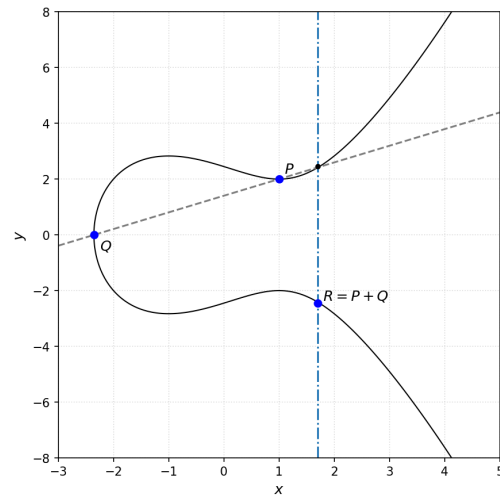


Figura (2): Representación geométrica de la suma de dos puntos  $P$  y  $Q$  sobre una curva elíptica

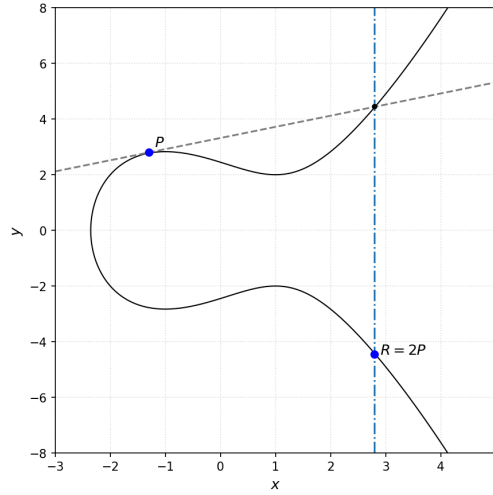


Figura (3): Representación geométrica del doblado de un punto  $P$  sobre una curva elíptica

### 2.3.3. Orden del grupo

Sea  $E$  una curva elíptica definida sobre un **campo finito**  $\mathbb{F}_p$ , el conjunto de puntos  $E(\mathbb{F}_p)$  junto al punto en el infinito  $\infty$  y la operación aditiva de suma de puntas definen un grupo **finito** abeliano  $(E(\mathbb{F}_p), +)$ .

El **Teorema de Hasse** establece que la orden del grupo  $(E(\mathbb{F}_p), +)$  satisface la relación:

$$|E(\mathbb{F}_p)| = p + 1 - t \quad \text{donde } |t| \leq 2\sqrt{p} \quad (15)$$

Esta desigualdad muestra que el orden del grupo de la curva elíptica no puede alejarse de  $p + 1$ . Si bien, el número exacto de puntos depende de la curva particular, su tamaño permanece dentro de un intervalo definido por  $p$ .

Dicha propiedad es de suma importancia, ya que permite anticipar el tamaño del grupo y seleccionar curvas cuyas subgrupos tengan orden suficientemente grande para resistir ataques conocidos.

A partir de esta definición, el problema del *logaritmo discreto* puede formularse sobre esta clase de grupos, los cuales ofrecen una estructura especialmente favorable para alcanzar un alto nivel de seguridad con tamaños de clave menores. En particular, para lograr un nivel de seguridad comparable, se requieren parámetros significativamente más pequeños que en el caso del grupo multiplicativo finito  $(\mathbb{F}_p^\times, \cdot)$ , lo que se traduce en una mayor eficiencia en términos de memoria, transmisión y cómputo.

## 2.4. Criptosistemas basados en curvas elípticas

### 2.4.1. EC Generación de claves

Sea  $E$  una curva elíptica definida sobre el campo finito  $\mathbb{F}_p$ . Sea  $P$  un punto en  $E(\mathbb{F}_p)$ , y asumiendo que  $P$  es de orden primo  $n$ . Entonces el subgrupo cíclico de  $E(\mathbb{F}_p)$  generado por  $P$  es:

$$\langle P \rangle = \{\infty, P, 2P, 3P, \dots, (n-1)P\} \quad (16)$$

Los parámetros públicos de dominio son: EL primo  $p$ , la ecuación de la curva elíptica  $E$ , y el punto  $P$  de orden primo  $n$ . Teniendo en cuenta dichos parámetros se procede a seleccionar

uniformemente al azar un entero  $d \in_R [1, n - 1]$  que será nuestra llave privada, y cuya correspondiente llave pública será  $Q = dP$

A continuación se presenta el algoritmo:

---

**Algoritmo 1.7:** Generación de par de llaves de EC

---

**Entrada:** Parámetros de dominio de curva elíptica  $(p, E, P, n)$

**Salida:** Llave pública  $Q$  y llave privada  $d$

1. Seleccionar  $d \in_R [1, n - 1]$
  2. Calcular  $Q = dP$
  3. **Regresar**  $(Q, d)$
- 

EL problema de determinar  $d$  dado los parámetros de dominio y  $Q$  es conocido como el problema del logaritmo discreto de curvas elípticas (**ECDLP**), una extensión que es posible gracias a que el problema formulado parte de un concepto de grupo general.

### 2.4.2. EC Esquema de Cifrado y Descifrado

El proceso de cifrado y descifrado es análogo al **DL**, variando que no usaremos un proceso multiplicativo sino aditivo. En este caso el texto plano  $m$  es representado como un punto  $M$ , por lo cual es encriptado de forma aditiva como  $kQ$  donde  $k$  es un entero seleccionado uniformemente al azar por el emisor. Él cual le enviara  $C_1 = kP$  y  $C_2 = M + kQ$  al receptor, quién deberá usar su llave privada  $d$  para calcular:

$$dC_1 = d(kP) = k(dP) = kQ \quad (17)$$

y así obtener  $M$  como:  $C_2 - kQ$

Sí alguien intentara obtener  $M$  necesitaría calcular  $kP$  a partir de los parámetros de dominio,  $P$  y  $C_1$ . Esto es el análogo del problema de Diffie-Hellman (**DHP**) para curvas elípticas, donde ahora nuestro problema se traduce a:

$$\log_P C_1 = k \quad (18)$$

A continuación se presenta tanto los algoritmos de encriptación como descifrado para **EC**:

---

**Algoritmo 1.8:** Encriptación básica de ElGamal por EC

---

**Entrada:** Parámetros de dominio de curva elíptica  $(p, E, P, n)$ , llave pública  $Q$ , texto plano  $m$

**Salida:** Texto cifrado  $(C_1, C_2)$

1. Representar el mensaje  $m$  como un punto  $M$  en  $E(\mathbb{F}_p)$
  2. Seleccionar  $k \in_R [1, n - 1]$
  3. Calcular  $C_1 = kP$
  4. Calcular  $C_2 = M + kQ$
  5. **Regresar**  $(C_1, C_2)$
-

---

**Algoritmo 1.9:** Descriptación básica de ElGamal por EC

---

**Entrada:** Parámetros de dominio  $(p, E, P, n)$ , llave privada  $d$ , texto cifrado  $(C_1, C_2)$

**Salida:** Texto plano  $m$

1. Calcular  $M = C_2 - dC_1$  y extraer  $m$  de  $M$

2. **Regresar**  $m$

---

**2.4.3. EC Esquema de Firmas (ECDSA)**

El algoritmo de firma digital (**ECDSA**) es análogo al (**DSA**). Por ende, el proceso es muy similar solo que ahora estamos trabajando con notación aditiva y curvas elípticas. El esquema (**ECDSA**) esta presente en las normas ANSI X9.62, **FIPS 186-5**, IEEE 1363-2000 y ISO/IEC 15946-2.

A continuación se presenta los algoritmos de generación y verificación de firma, considerando una forma abreviada de los parámetros de dominio, usando simplemente lo visto anteriormente.

---

**Algoritmo 1.10:** Generación de firma **ECDSA**

---

**Entrada:** Parámetros de dominio  $(p, E, P, n)$ , llave privada  $d$ , mensaje  $m$

**Salida:** Firma  $(r, s)$

1. Seleccionar  $k \in_R [1, n - 1]$

2. Calcular  $kP = (x_1, y_1)$

3. Calcular  $r = x_1 \bmod n$ . Si  $r = 0$  regresar al paso 1.

4. Calcular  $e = H(m)$

5. Calcular  $s = k^{-1}(e + dr) \bmod n$ . Si  $s = 0$  regresar al paso 1.

6. **Regresar**  $(r, s)$

---

---

**Algoritmo 1.11:** Verificación de firma **ECDSA**

---

**Entrada:** Parámetros de dominio  $(p, E, P, n)$ , llave pública  $Q$ , mensaje  $m$ , firma  $(r, s)$

**Salida:** Aceptación o rechazo de la firma

1. Verificar que  $r$  y  $s$  sean enteros en el intervalo  $[1, n - 1]$ . Si alguna verificación falla, regresar («Rechazar la firma»)

2. Calcular  $e = H(m)$

3. Calcular  $w = s^{-1} \bmod n$

4. Calcular  $u_1 = ew \bmod n$  y  $u_2 = rw \bmod n$

5. Calcular  $X = u_1P + u_2Q$

6. Si  $X = \infty$ , regresar («Rechazar la firma»)

7. Calcular  $r' = x_1 \bmod n$ , donde  $X = (x_1, x_2)$

8. Si  $r' = r$ , **Regresar** (Aceptar la firma);

Si no, **Regresar** (Rechazar la firma).

---

En **ECDSA** para demostrar porque se puede verificar la firma, de igual forma que en **DSA** partimos de la generación de la firma  $s$ :

$$s = k^{-1}(e + dr) \bmod n$$

La cual es equivalente a:

$$k \equiv s^{-1}e + s(-1)dr \bmod n$$

Sea  $w = s^{-1} \bmod n$  y reemplazando, tenemos:

$$u_1 = ew \bmod n = s^{-1}e \bmod n, \quad u_2 = rw \bmod n = s^{-1}r \bmod n \quad (19)$$

Por lo tanto:

$$k \equiv u_1 + u_2d \pmod{n} \quad (20)$$

Multiplicando ambos lados por el punto generador  $P$ , tenemos:

$$kP = u_1P + u_2(dP) \quad (21)$$

Como la llave pública es  $Q = dP$ , tenemos:

$$kP = u_1P + u_2Q \quad (22)$$

El lado izquierdo es el punto que el firmante usó para producir  $r$ ; el lado derecho es el punto  $X$  que calcula la persona que desea verificar la firma digital. Finalmente tenemos así que sus coordenadas de  $X$  coinciden por ende se verifica que  $r' = r$

#### 2.4.4. EC-KCDSA Esquema de Firmas

**EC-KCDSA** a diferencia de **ECDSA** que tiene origen americano y está estandarizado por el **NIST**, surge de Corea del Sur y sigue la norma **ISO/IEC 15946-2**.

En **ECDSA** la llave pública es  $Q = dP$ . En **EC-KCDSA** se define como:  $Q = d^{-1}P$ . Por lo cual su proceso de generación y verificación de firmas no necesitara de calcular inversos modulares. En contraste **ECDSA** requiere calcular  $k^{-1} \bmod n$  y  $s^{-1} \bmod n$ .

EL proceso de generación de firma del **EC-KCDSA** es:

Se selecciona el  $k$  y calcular  $kP$  es de igual forma que el **ECDSA**. Para calcular  $r$  en este caso se **hashea** directamente. Lo novedoso es que se calcula un  $e$  como resultado de **hashear** a la vez los datos de certificación del firmante con el mensaje  $m$ . Posteriormente se calcula  $w$  aplicando un operación **XOR** entre  $r$  y  $e$ .

Finalmente se calcula  $s$  multiplicando por  $d$  directamente, sin tener que calcular inversas:

$$s = d(k - \overline{w}) \bmod n \quad (23)$$

A continuación se presentan los algoritmos de generación y verificación de firma, considerando una forma abreviada de los parámetros de dominio, para no entrar en mayor detalle.

---

#### Algoritmo 1.12: Generación de firma **EC-KCDSA**

---

**Entrada:** Parámetros de dominio  $(p, E, P, n)$ , llave privada  $d$ , datos de certificación hasheados  $h_{\text{cert}}$ , mensaje  $m$

**Salida:** Firma  $(r, s)$

1. Seleccionar  $k \in_R [1, n - 1]$

---



---

**Algoritmo 1.12:** Generación de firma **EC-KCDSA**

---

2. Calcular  $kP = (x_1, y_1)$
  3. Calcular  $r = H(x_1)$
  4. Calcular  $e = H(h_{\text{cert}}, m)$
  5. Calcular  $w = r \oplus e$  y convertir  $w$  a entero  $\bar{w}$
  6. Si  $\bar{w} \geq n$ , entonces  $\bar{w} \leftarrow \bar{w} - n$
  7. Calcular  $s = d(k - \bar{w}) \bmod n$ . Si  $s = 0$  regresar al paso 1
  8. **Regresar**  $(r, s)$
- 

---

**Algoritmo 1.13:** Verificación de firma **EC-KCDSA**

---

**Entrada:** Parámetros de dominio  $(p, E, P, n)$ , llave pública  $Q$ , datos de certificación hashados  $h_{\text{cert}}$ , mensaje  $m$ , firma  $(r, s)$

**Salida:** Aceptación o rechazo de la firma

1. Verificar que la longitud en bits de  $r$  sea a lo más  $l_H$  y que  $s \in [1, n - 1]$ . Si alguna falla, **Regresar**(Rechazar la firma)
  2. Calcular  $e = H(h_{\text{cert}}, m)$
  3. Calcular  $w = r \oplus e$  y convertir  $w$  a entero  $\bar{w}$
  4. Si  $\bar{w} \geq n$ , entonces  $\bar{w} \leftarrow \bar{w} - n$
  5. Calcular  $X = sQ + \bar{w}P$
  6. Calcular  $v = H(x_1)$ , donde  $x_1$  es la coordenada  $x$  de  $X$
  7. Si  $v = r$ , **Regresar**(Aceptar la firma);  
Si no, **Regresar**(Rechazar la firma)
- 

En **EC-KCDSA** para demostrar porque se puede verificar la firma partimos de lo siguiente:  
 $s = d(k - \bar{w}) \bmod n$ , Despejando  $k$ :

$$sd^{-1} = k - \bar{w} \bmod n \quad (24)$$

$$k \equiv sd^{-1} + \bar{w} \bmod n \quad (25)$$

Multiplicando ambos lados por  $P$ :

$$kP = sd^{-1}P + \bar{w}P \quad (26)$$

Como la llave pública es  $Q = d^{-1}P$ :

$$kP = sQ + \bar{w}P = X \quad (27)$$

La persona que desea verificar la firma, al realizar este proceso reconstruye exactamente el mismo punto  $kP$  que usó el firmante.

Por lo tanto  $v = H(x_1) = r$

Se recalca nuevamente que para este proceso no fue necesario calcular ningún inverso modular, a diferencia del **ECDSA**.

## 3. Análisis de seguridad

### 3.1. Seguridad GMR

Un sistema de firmas digitales es *GMR*-seguro cuando es resistente a **ataques de falsificación de firmas bajo mensajes escogidos adaptativos**. En otras palabras, dado un oráculo que reciba mensajes arbitrarios y genere firmas digitales válidas, un atacante no puede abusar de este para falsificar una firma válida no generada por el oráculo.

#### 3.1.1. ECDSA

El protocolo ECDSA requiere que el problema de logaritmo discreto de la curva  $\langle P \rangle$  sea irresoluble y que la función *hash* para calcular  $e$  sea criptográficamente segura (resistente a ataques de preimagen y colisiones) para que se demuestre *GMR*-seguro. Sin embargo, no se ha demostrado que estas condiciones sean suficientes. Teóricamente, llega a ser *GMR*-seguro cuando se usa un modelo de grupo genérico en lugar de  $\langle P \rangle$ . Es decir, si se tiene un modelo que abstraiga los aspectos físicos y geométricos del grupo, tomando a la curva como una «caja negra» u oráculo que realice operaciones deterministas pero sin conocer cómo las realiza, el algoritmo se prueba completamente seguro ante los ataques de mensajes escogidos adaptativos.

Sin embargo, estos modelos abstractos no se traducen a las implementaciones reales concretas de curvas elípticas, por lo que no se llega a una seguridad *GMR* verdadera. Aun así, inspira la seguridad suficiente para su uso actual.

#### 3.1.2. EC-KCDSA

De modo similar a ECDSA, el esquema EC-KCDSA se demuestra *GMR*-seguro bajo un par de suposiciones: el problema del logaritmo discreto es irresoluble y que la función de hash utilizada sea completamente aleatoria (en caso se usen funciones distintas, solo se requiere que  $H_1$  sea aleatoria mientras que  $H_2$  sea resistente a colisiones). Nuevamente, esta aplicación teórica de funciones hash puramente aleatorias no es completamente aplicable a la realidad. Sin embargo, difícilmente un adversario podría aprovechar las propiedades no aleatorias de este sistema, por lo que se puede considerar lo suficientemente seguro pese a no ser *GMR*-seguro.

### 3.2. Verificación de $r$ y $s$

#### 3.2.1. ECDSA

Se requiere que ambos valores se encuentren en el intervalo  $[1, n - 1]$  para el correcto funcionamiento del algoritmo. Caso contrario, se pueden falsificar firmas bajo los supuestos de que se use un punto base  $P = (0, \sqrt{b})$  en la curva  $y^2 = x^3 + ax + b$  (siendo  $b$  un primo de orden grande), resultando que una firma válida para un mensaje  $m$  sea  $(r = 0, s = e)$ . Es decir, se corre el riesgo de poder generar firmas triviales.

#### 3.2.2. EC-KCDSA

De forma similar, se verifica que  $s$  se encuentre en el intervalo  $[1, n - 1]$  y que  $r$  tenga a lo más  $l_H$  bits. La verificación de  $r$  se debe a que un adversario podría interceptar una firma válida  $(r, s)$  y extender los bits de  $r$  y generar un  $r'$ . Si el software del algoritmo ignora estos bits y verifica la validez de la firma, entonces el atacante efectivamente ha falsificado una firma  $(r', s)$ .

### 3.3. Secreto $k$

#### 3.3.1. ECDSA

El secreto generado  $k$  debe cumplir con ser aleatorio y privado para evitar ataques al sistema ECDSA. Si el atacante obtiene conocimiento de uno de estos secretos  $k$ , se puede recuperar la clave privada simplemente con  $d = r^{-1}(ks - e) \bmod n$ , con  $e = H(m)$ . Incluso conociendo solo los primeros bits de un conjunto enorme de mensajes, un atacante es capaz de derivar la llave secreta mediante ataques de canal lateral (midiendo tiempos de respuesta, cambios de voltaje, etc).

Si el secreto  $k$ , en cambio, es reutilizado en múltiples firmas, este puede ser obtenido con firmas sucesivas utilizando la fórmula  $k \equiv (s_1 - s_2)^{-1}(e_1 - e_2) \bmod n$ . Como consecuencia, se deriva la llave secreta con la fórmula utilizada anteriormente.

#### 3.3.2. EC-KCDSA

Lo mismo se cumple para este sistema:  $k$  debe ser único y secreto. Para el caso en el que no sea secreto, se puede recuperar la llave privada con  $d \equiv s(\bar{w})^{-1} \bmod n$ . Reusar  $k$  permite obtener  $d$  con la fórmula  $d = \frac{s_1 - s_2}{\bar{w}_2 - \bar{w}_1}$

## 4. Implementacion en codigo

### 4.1. ECDSA

### 4.2. EC-KCDSA

## 5. Conclusiones

### 5.1. Conclusión de resistencia

Las consideraciones y posibles vulnerabilidades detalladas en esta sección, por lo general, dan por sentado que el problema del logaritmo discreto sea lo suficientemente difícil como para que no tenga posibilidad de resolverlo en el futuro cercano (sin tomar en cuenta el riesgo de la computación cuántica).

Pese a todas las consideraciones en ambos sistemas de firmado digital ECDSA y EC-KCDSA, las implementaciones actuales de ambos sistemas son lo suficientemente cercanas a las consideraciones teóricas necesarias para su seguridad (por ejemplo, se usan funciones hash y sistemas de generación deterministas de  $k$  suficientemente aleatorios en la práctica para que sea posible un ataque que aproveche su determinismo). Su constante escrutinio e implementaciones detalladas y rigurosamente testeadas, aunque no den seguridad certera, dan la suficiente confianza para su uso en sistemas que requieran firmados digitales robustos y seguros.